

# COMS/SE 4170 Software Testing Spring 2026

Combination Strategies  
(Chapters 6.2-6.4)

1

## Quick Recap Input Partitioning

2

### Second Characterization of triang()'s Inputs

| Characteristic                 | $b_1$          | $b_2$      | $b_3$      | $b_4$       |
|--------------------------------|----------------|------------|------------|-------------|
| $q_1$ = "Refinement of $q$ "   | greater than 1 | equal to 1 | equal to 0 | less than 0 |
| $q_2$ = "Refinement of $q$ "   | greater than 1 | equal to 1 | equal to 0 | less than 0 |
| $q_3$ = "Refinement of $q_3$ " | greater than 1 | equal to 1 | equal to 0 | less than 0 |

- A maximum of  $4 \times 4 \times 4 = 64$  tests
- Complete because the inputs are integers (0 . . 1)

### Possible values for partition $q_1$

| Characteristic | $b_1$ | $b_2$ | $b_3$ | $b_4$ |
|----------------|-------|-------|-------|-------|
| Side I         | 2     | 1     | 0     | -1    |

Test boundary conditions

Introduction to Software Testing, Edition 2 (Ch 6)

© Ammann & Offutt

3

3

## Part II Chapter 6

Combination Strategies

4

## Step 4 – Choosing Combinations of Values (6.2)

- Once characteristics and partitions are defined, the next step is to choose test values
- We use criteria – to choose effective subsets

**All Combinations (ACoC):** All combinations of blocks from all characteristics must be used.

- Number of tests is the product of the number of blocks in each characteristic :  $\prod_{i=1}^Q (B_i)$
- The second characterization of `triang()` results in  $4*4*4 = 64$  tests
  - Too many ?

Introduction to Software Testing, Edition 2 (Ch 6)

© Ammann & Offutt

5

5

## ISP Criteria – All Combinations

- Consider the “second characterization” of `Triang` as given before:

| Characteristic                                   | b <sub>1</sub> | b <sub>2</sub> | b <sub>3</sub> | b <sub>4</sub> |
|--------------------------------------------------|----------------|----------------|----------------|----------------|
| q <sub>1</sub> = “Refinement of q <sub>1</sub> ” | greater than 1 | equal to 1     | equal to 0     | less than 0    |
| q <sub>2</sub> = “Refinement of q <sub>2</sub> ” | greater than 1 | equal to 1     | equal to 0     | less than 0    |
| q <sub>3</sub> = “Refinement of q <sub>3</sub> ” | greater than 1 | equal to 1     | equal to 0     | less than 0    |

- For convenience, we relabel the blocks:

| Characteristic | b <sub>1</sub> | b <sub>2</sub> | b <sub>3</sub> | b <sub>4</sub> |
|----------------|----------------|----------------|----------------|----------------|
| A              | A1             | A2             | A3             | A4             |
| B              | B1             | B2             | B3             | B4             |
| C              | C1             | C2             | C3             | C4             |

Introduction to Software Testing, Edition 2 (Ch 6)

© Ammann & Offutt

6

6

## ISP Criteria – ACoC Tests

|          |          |          |          |
|----------|----------|----------|----------|
| A1 B1 C1 | A2 B1 C1 | A3 B1 C1 | A4 B1 C1 |
| A1 B1 C2 | A2 B1 C2 | A3 B1 C2 | A4 B1 C2 |
| A1 B1 C3 | A2 B1 C3 | A3 B1 C3 | A4 B1 C3 |
| A1 B1 C4 | A2 B1 C4 | A3 B1 C4 | A4 B1 C4 |
| A1 B2 C1 | A2 B2 C1 | A3 B2 C1 | A4 B2 C1 |
| A1 B2 C2 | A2 B2 C2 | A3 B2 C2 | A4 B2 C2 |
| A1 B2 C3 | A2 B2 C3 | A3 B2 C3 | A4 B2 C3 |
| A1 B2 C4 | A2 B2 C4 | A3 B2 C4 | A4 B2 C4 |
| A1 B3 C1 | A2 B3 C1 | A3 B3 C1 | A4 B3 C1 |
| A1 B3 C2 | A2 B3 C2 | A3 B3 C2 | A4 B3 C2 |
| A1 B3 C3 | A2 B3 C3 | A3 B3 C3 | A4 B3 C3 |
| A1 B3 C4 | A2 B3 C4 | A3 B3 C4 | A4 B3 C4 |
| A1 B4 C1 | A2 B4 C1 | A3 B4 C1 | A4 B4 C1 |
| A1 B4 C2 | A2 B4 C2 | A3 B4 C2 | A4 B4 C2 |
| A1 B4 C3 | A2 B4 C3 | A3 B4 C3 | A4 B4 C3 |
| A1 B4 C4 | A2 B4 C4 | A3 B4 C4 | A4 B4 C4 |

ACoC yields  
 $4*4*4 = 64$  tests  
for `Triang`!

This is almost  
certainly more  
than we need

Only 8 are valid  
(all sides greater  
than zero)

Introduction to Software Testing, Edition 2 (Ch 6)

© Ammann & Offutt

7

7

## Functionality-Based *IDM*—*triang()*

- First two characterizations are based on syntax—parameters and their type
- A semantic level characterization could use the fact that the three

| Characteristic                              | b <sub>1</sub> | b <sub>2</sub> | b <sub>3</sub> | b <sub>4</sub> |
|---------------------------------------------|----------------|----------------|----------------|----------------|
| q <sub>1</sub> = “Geometric Classification” | scalene        | isosceles      | equilateral    | invalid        |

### Correct Geometric Characterization of *triang()*'s Inputs

| Characteristic                              | b <sub>1</sub> | b <sub>2</sub>             | b <sub>3</sub> | b <sub>4</sub> |
|---------------------------------------------|----------------|----------------------------|----------------|----------------|
| q <sub>1</sub> = “Geometric Classification” | scalene        | isosceles, not equilateral | equilateral    | invalid        |

8

## ISP Criteria – Each Choice

- 64 tests for triang() is almost certainly way too

**Each Choice Coverage (ECC)** : One value from each block for

- each characteristic must be used in at least one test case.

should try at least one value from each block

- Number of tests is the number of blocks in the largest characteristic :  $\text{Max}_{i=1}^Q (B_i)$

For *triang()* : A1, B1, C1  
A2, B2, C2  
A3, B3, C3  
A4, B4, C4

Substituting values: 2, 2, 2  
1, 1, 1  
0, 0, 0  
-1, -1, -1

Introduction to Software Testing, Edition 2 (Ch 6)

© Ammann & Offutt

9

9

## ISP Criteria – Pair-Wise

- Each choice yields few tests—cheap but maybe ineffective

**Pair-Wise Coverage (PWC)** : A value from each block for each characteristic must be combined with a value from every block for each other characteristic.

- Number of tests is at least the product of two largest characteristics  $(\text{Max}_{i=1}^Q (B_i)) * (\text{Max}_{j=1, j \neq i}^Q (B_j))$

For *triang()* : A1, B1, C1 A1, B2, C2 A1, B3, C3 A1, B4, C4  
A2, B1, C2 A2, B2, C3 A2, B3, C4 A2, B4, C1  
A3, B1, C3 A3, B2, C4 A3, B3, C1 A3, B4, C2  
A4, B1, C4 A4, B2, C1 A4, B3, C2 A4, B4, C3

Introduction to Software Testing, Edition 2 (Ch 6)

© Ammann & Offutt

10

10

## ISP Criteria –T-Wise

- A natural extension is to require combinations

**t-Wise Coverage (TWC)** : A value from each block for each group of t characteristics must be combined.

- Number of tests is at least the product of t largest characteristics
- If all characteristics are the same size, the formula is

$$(\text{Max}_{i=1}^Q (B_i))^t$$

- If t is the number of characteristics Q, then all combinations
- That is ... Q-wise = AC

Introduction to Software Testing, Edition 2 (Ch 6)

© Ammann & Offutt

11

11

## ISP Criteria – Base Choice

- Testers sometimes recognize that certain values are important

**Base Choice Coverage (BCC)** : A base choice block is chosen for each characteristic, and a base test is formed by using the base choice for each characteristic. Subsequent tests are chosen by holding all but one base choice constant and using each non-base choice in each other characteristic.

- Number of tests is one base test + one test for each other block  $1 + \sum_{i=1}^Q (B_i - 1)$

For *triang()* : Base A1, B1, C1 A1, B1, C2 A1, B2, C1 A2, B1, C1  
A1, B1, C3 A1, B3, C1 A3, B1, C1  
A1, B1, C4 A1, B4, C1 A4, B1, C1

Introduction to Software Testing, Edition 2 (Ch 6)

© Ammann & Offutt

12

12

## Base Choice Notes

- The base **test must be feasible**
  - That is, all base choices must be compatible
- Base choices can be
  - Most likely from an end-use point of view
  - Simplest
  - Smallest
  - First in some ordering
- Happy path tests often make good base choices
- The base choice is a crucial design decision
  - Test designers should document why the choices were made

13

## ISP Criteria – Multiple Base Choice

**Multiple Base Choice Coverage (MBCC)** : At least one, and possibly more, base choice blocks are chosen for each characteristic, and base tests are formed by using each base choice for each characteristic at least once. Subsequent tests are chosen by holding all but one base choice constant for each base test and using each non-base choice in each other characteristic.

- If  $M$  base tests and  $m_i$  base choices for each characteristic:

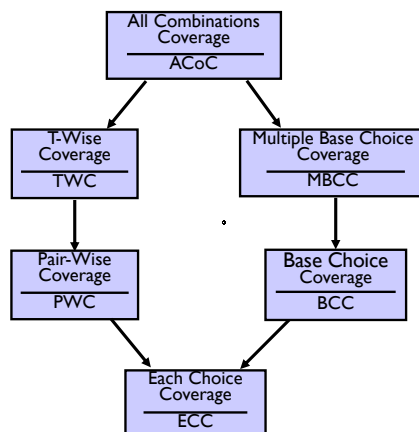
$$M + \sum_{i=1}^Q (M * (B_i - m_i))$$

For triang() : Bases

|            |            |            |            |
|------------|------------|------------|------------|
| A1, B1, C1 | A1, B1, C3 | A1, B3, C1 | A3, B1, C1 |
|            | A1, B1, C4 | A1, B4, C1 | A4, B1, C1 |
| A2, B2, C2 | A2, B2, C3 | A2, B3, C2 | A3, B2, C2 |
|            | A2, B2, C4 | A2, B4, C2 | A4, B2, C2 |

14

## ISP Coverage Criteria Subsumption



15

## Constraints Among Characteristics (6.3)

- Some combinations of blocks are infeasible
  - “less than zero” and “scalene” ... not possible at the same time
- These are represented as constraints among blocks
- Two general types of constraints
  - A block from one characteristic cannot be combined with a specific block from another
  - A block from one characteristic can ONLY BE combined with a specific block from another characteristic

16

### Constraints Among Characteristics (6.3)

- Handling constraints depends on the criterion used
  - ACC, PWC, TWC : Drop the infeasible pairs
  - BCC, MBCC : Change a value to another non-base choice to find a feasible combination